ETL Design & Rerun-Safe Architecture Documentation

---

1. ETL Architecture Overview

The ETL pipeline is structured into three logical layers:

- **STG (Staging Layer)**
- **DIM (Dimension Layer)**
- **FACT (Fact Layer)**

Execution is orchestrated through:

- Layer-based worklets
- A master workflow
- Centralized audit logging

2. Rerun-Safe Design Using Business Date

2.1 Business Date Parameter

A global parameter controls the logical processing date:

```
[Global]
$$BUSINESS_DATE = YYYY-MM-DD
```

This parameter represents the **business processing date**, independent of the system date, and enables **controlled reruns** for any specific day.

2.2 Rerun Strategy in the STG Layer

Before loading data into staging tables, a **Pre-SQL cleanup** is executed:

```
DELETE FROM STG_TABLE
WHERE CAST(LOADDATE AS DATE) = CAST('$$BUSINESS_DATE' AS DATE)
AND LOADDATE <  DATEADD(DAY, 1, CAST('$$BUSINESS_DATE' AS DATE));
```

This ensures that:

- Only data for the selected business dates is removed

- Historical data is preserved
- Duplicate or partial loads are prevented
- STG loads are fully rerun-safe

---

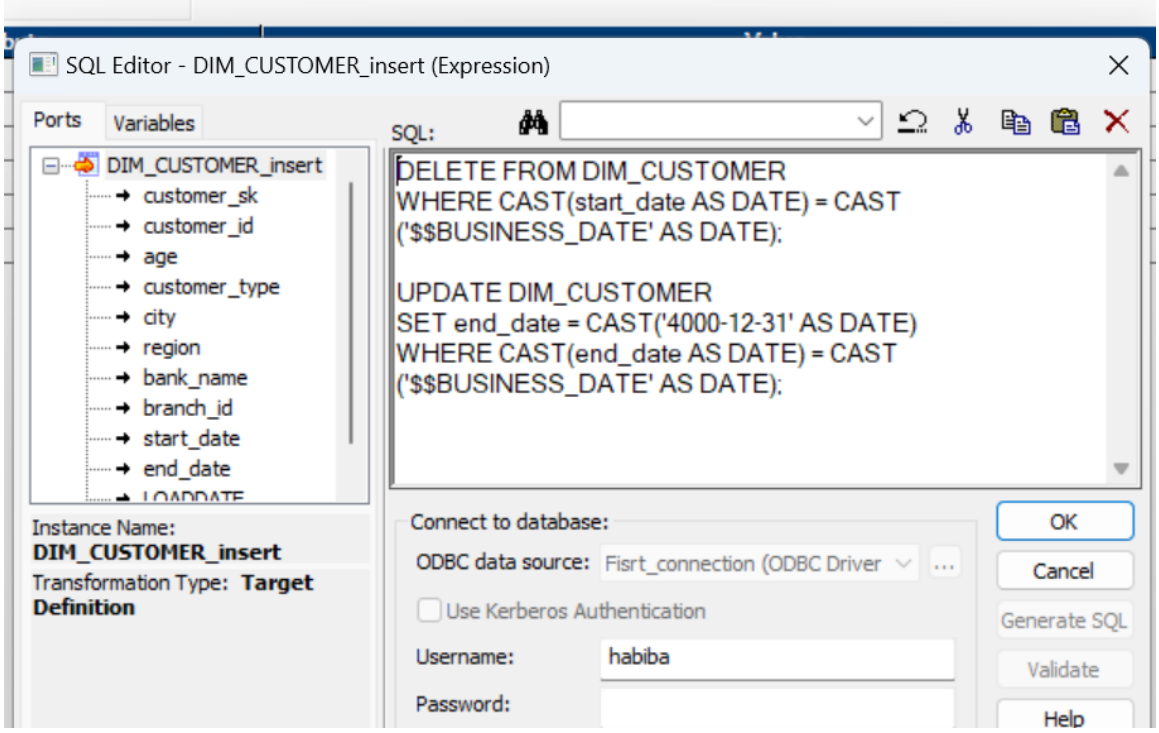
3. SCD Rerun Handling Summary (Type 1 & Type 2)

Both **SCD Type 1** and **SCD Type 2** dimensions are implemented using a **business-date-driven rerun strategy**.

Before each dimension load, **Pre-SQL cleanup and rollback logic** ensure that any data affected by a rerun is safely removed or restored.

- **SCD Type 1** reruns overwrite existing dimension records to reflect the most recent values.
- **SCD Type 2** reruns restore the dimension to a consistent historical state by:
  - Removing newly inserted records
  - Reopening previously closed records

This approach guarantees **rerun safety, data consistency, and full recoverability** of dimension data for any business date.



4. Rerun Strategy for FACT Tables

Fact tables follow the same **business-date-driven rerun principle** as the staging layer.

Before loading fact data, a **Pre-SQL cleanup** is applied:

```
DELETE FROM FACT_TABLE
WHERE CAST(LOADDATE AS DATE) = CAST('$$BUSINESS_DATE' AS DATE)
AND LOADDATE <  DATEADD(DAY, 1, CAST('$$BUSINESS_DATE' AS DATE));
```

This ensures that:

- Fact data is never duplicated on reruns
- Facts are always rebuilt from validated STG and DIM data

Fact loading is executed **only after successful completion of STG and DIM layers**.

5. Worklet-Based Design

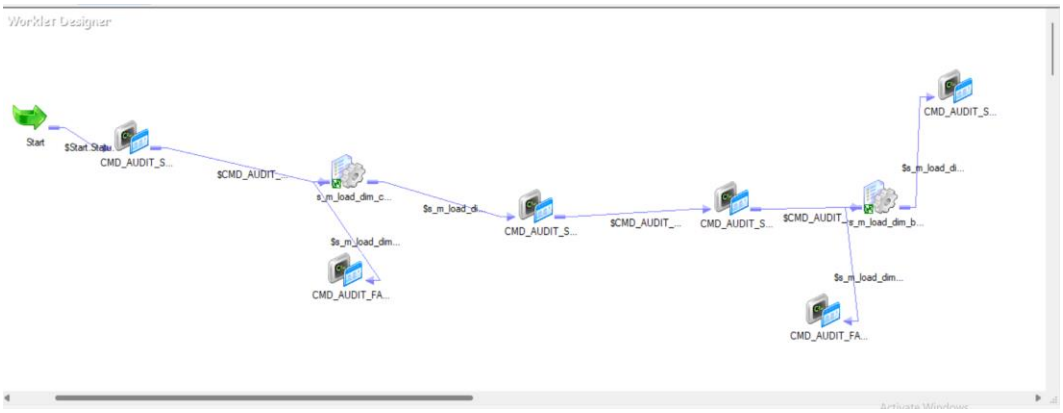
5.1 Standard Worklet Structure

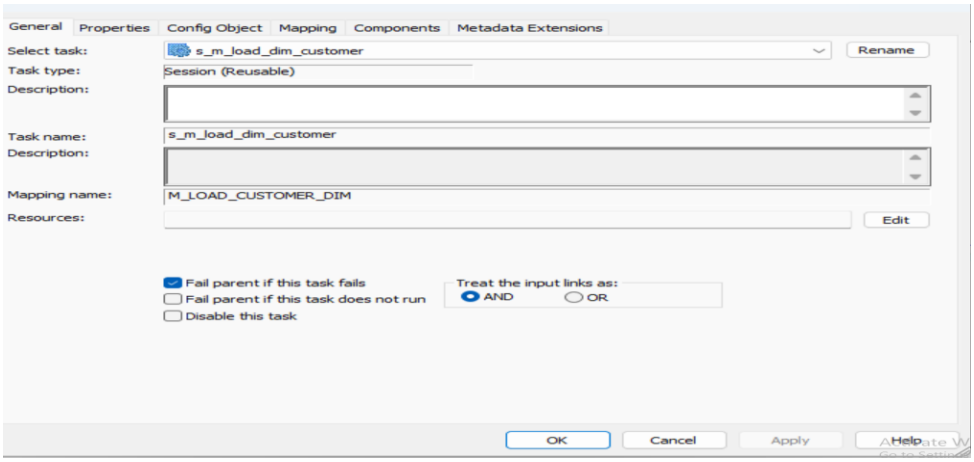
All worklets (**STG, DIM, FACT**) follow the same standardized structure:

1. Audit **START** command task
2. Session execution
3. Audit **SUCCESS** command task
4. Audit **FAILED** command task (on failure)

Any session failure causes:

- The worklet to fail
- The workflow to fail





---

5. Workflow Orchestration

5.1 Master Workflow Design

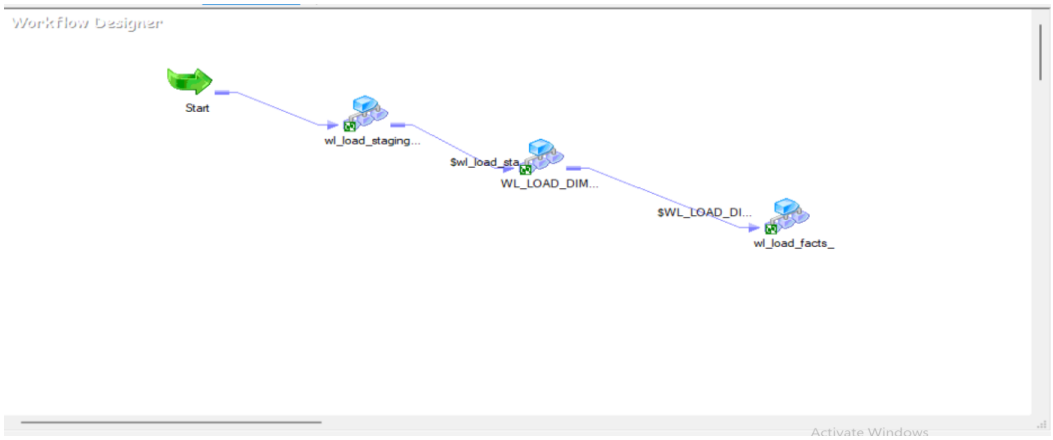
A single master workflow controls execution order:

```
STG Worklet → DIM Worklet → FACT Worklet
```

Execution rules:

- DIM does **not** run if STG fails
- FACT does **not** run if DIM fails
- No downstream processing occurs after a failure

This guarantees **correct load sequencing and data integrity**.



6. Audit Table Technique

6.1 Purpose of the Audit Table

The audit table provides:

- Execution visibility per session
- Tracking of STARTED / SUCCESS / FAILED statuses
- Last success date for every mapping so we can roll back to this date

Results | Messages

	MAPPING_NAME	STATUS	START_DATE	END_DATE	LAST_SUCCESS_RUN
1	M_LOAD_STG_CUSTOMER	FAILED	2026-02-04 00:00:00.000	2026-02-04 03:29:07.150	2026-02-04 03:24:36.157
2	M_LOAD_STG_TRANSACTION	SUCCESS	2026-02-04 00:00:00.000	2026-02-04 03:25:40.803	2026-02-04 03:25:40.803
3	M_LOAD_CUSTOMER_DIM	SUCCESS	2026-02-04 00:00:00.000	2026-02-04 03:26:12.543	2026-02-04 03:26:12.543
4	M_LOAD_BANK_DIM_	SUCCESS	2026-02-04 00:00:00.000	2026-02-04 03:26:44.660	2026-02-04 03:26:44.660
5	M_LOAD_FACT_TRANSACTION	SUCCESS	2026-02-04 00:00:00.000	2026-02-04 03:27:17.620	2026-02-04 03:27:17.620

Disconnected.